Lab 5

MSDS684_S70_Reinforcement Learning

Maryi Tatiana Palacios Giraldo

Regis University

April 2026

Project Overview

This lab explores the transition from tabular reinforcement learning methods to function approximation. In environments like MountainCar, the state space is continuous, making tabular methods impractical due to the infinite number of possible states.

To address this, I implemented linear function approximation using tile coding, which discretizes the continuous space into overlapping regions and allows generalization across similar states.

The objective was to apply semi-gradient SARSA, an on-policy control method, using feature vectors instead of state tables.

Deliverables

GitHub Repository:

https://github.com/MPalacios-Analytics/DataScience_Projects/tree/91f9c873f767025a209ca38b01085d079c12c75b/06_Reinforcement_Learning/Lab_5

Implementation Summary

The environment used was *MountainCar-v0*, where the state consists of position and velocity within continuous bounds.

A custom tile coding feature constructor was implemented using NumPy:

- 8 tilings
- 8×8 tiles per tiling
- Offsets applied to create overlapping grids

Each state is converted into a sparse binary feature vector, where only a small number of features are active.

The action-value function was approximated as:

$$Q(s, a) = w_a \cdot x(s)$$

where:

- w_a is the weight vector for action a
- $x(s)$ is the feature vector

Weights were updated using the semi-gradient SARSA rule:

$$w \leftarrow w + \alpha(r + \gamma Q(s', a') - Q(s, a))x$$

Key Results & Analysis

The agent showed gradual improvement over episodes, with a decreasing number of steps required to reach the goal.

Mountain Car is challenging because:

- The state space is continuous
- The agent must learn to move away from the goal to gain momentum
- Rewards are sparse and mostly negative

Tabular methods would require infinitely many states, making learning impossible.

Effect of Tile Coding

Tile coding enabled:

- Generalization across nearby states
- Faster learning compared to naive discretization

Increasing the number of tilings:

- Improves learning accuracy
- Increases computational cost

Smaller tiles:

- More precise representation
- Slower learning due to less generalization

An additional experiment comparing 4 vs 8 tilings was conducted. The results showed that 8 tilings led to faster and more stable learning, while 4 tilings resulted in slower convergence due to a coarser representation of the state space.

Learning Behavior

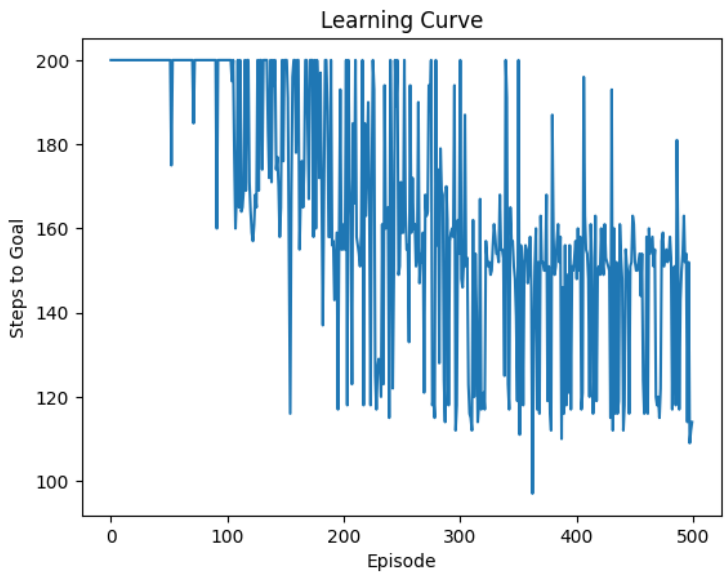
- Early training: high steps (random behavior)

- Mid training: agent discovers momentum strategy
- Later training: more stable convergence

Visualizations

1. Learning Curve

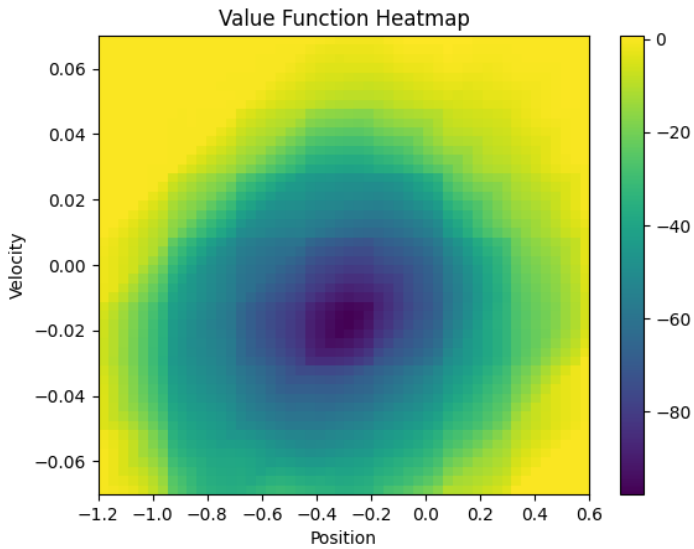
Shows reduction in steps per episode, indicating improved policy.



The learning curve shows a gradual decrease in steps per episode, indicating that the agent is improving its policy over time. Early episodes have high variance due to exploration, while later episodes stabilize as the agent learns to build momentum more efficiently.

2. Value Function Heatmap

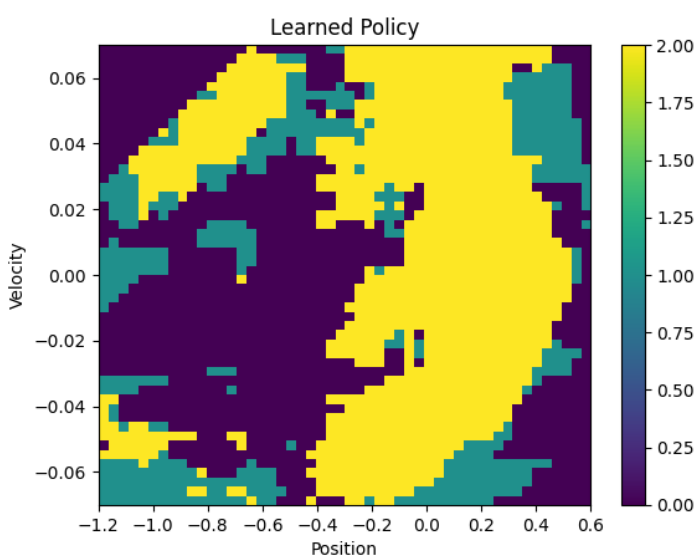
Displays max Q-values across position and velocity space.
Higher values appear near the goal region.



The heatmap shows lower values (darker regions) in areas where the agent struggles to reach the goal and higher values closer to the goal position. The smooth gradient indicates that tile coding successfully generalizes value estimates across similar states.

3. Policy Visualization

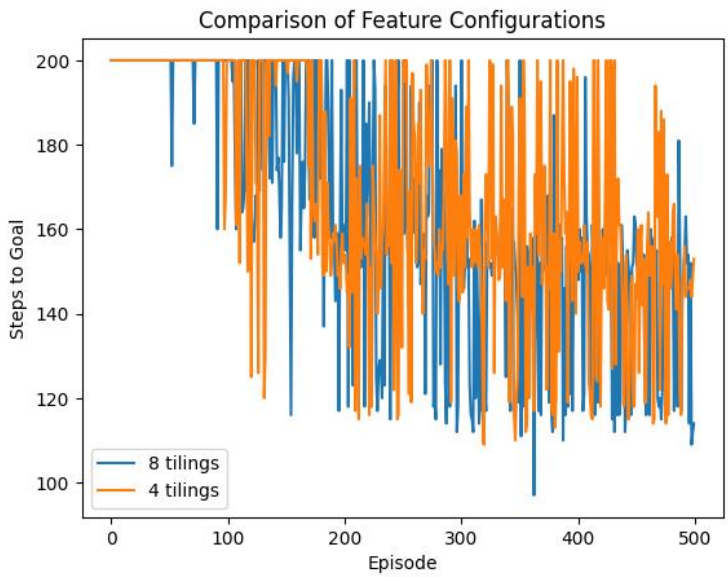
Shows the action selected by the agent across the state space, illustrating how the learned policy varies with position and velocity.



The policy visualization reveals distinct regions where specific actions are preferred. The agent learns to push left in low-momentum states and right when gaining velocity, demonstrating the learned strategy of building momentum before reaching the goal.

4. Comparison of Feature Configurations

Compares learning performance using 4 vs 8 tilings, showing that higher tiling resolution leads to faster and more stable convergence.



The comparison shows that using 8 tilings results in faster and more stable convergence compared to 4 tilings. The higher-resolution feature representation allows more accurate value estimation, improving learning efficiency.

AI Use Reflection

Initial Interaction:

I used AI to understand how to implement tile coding for continuous states.

I asked: "How can I convert a continuous state into tile indices using NumPy?"

The AI suggested the following approach:

```
ratios = (state - low) / tile_width
indices = np.floor(ratios).astype(int)
indices = np.clip(indices, 0, tiles - 1)
```

This helped me correctly map continuous values to discrete tile indices.

Use During Development:

I used AI to debug issues in my implementation.

For example, I asked: "Why are my tile indices going out of bounds?"

The AI suggested using clipping:

```
indices = np.clip(indices, 0, tiles - 1)
```

which fixed the issue.

I also asked: "Is my semi-gradient SARSA update correct?"

The AI suggested the following update rule:

```
delta = reward + gamma * q_next - q_current
weights[action] += alpha * delta * features
```

After comparing this with my implementation, I confirmed that my update was correct.

Critical Evaluation:

While AI provided useful guidance, some explanations were high-level and required careful validation.

For example, although the AI described tile coding conceptually, it did not initially emphasize the importance of properly scaling and clipping the state space. I identified this issue when I observed inconsistent learning behavior and invalid indices during execution.

This demonstrated that AI suggestions must be tested and aligned with the mathematical formulation of the algorithm.

Learning Reflection

Through this process, I developed a deeper understanding of how function approximation works in reinforcement learning.

Implementing tile coding helped me understand how continuous state spaces can be transformed into feature representations that allow generalization. Additionally, working with semi-gradient SARSA clarified how updates rely on the TD error and why the target is treated as fixed during learning.

This experience reinforced that small implementation details, such as feature scaling and index calculation, can significantly impact learning performance. I also

learned that AI tools are most effective when used iteratively and critically evaluated against theoretical expectations.

Speaker Notes

- This project focuses on solving the MountainCar problem using function approximation instead of tabular methods.
- MountainCar is challenging because the state space is continuous, meaning there are infinitely many possible states, and the agent must learn to build momentum by moving away from the goal first.
- To handle this, I implemented tile coding, which divides the state space into overlapping grids and allows generalization across similar states.
- Each state is represented as a sparse feature vector, and the action-value function is approximated using a linear combination of these features.
- I used semi-gradient SARSA, an on-policy method, to update the weights based on the Temporal-Difference error.
- The results show that the agent improves over time, reducing the number of steps needed to reach the goal as it learns a better policy.
- I also compared different feature configurations, showing that increasing the number of tilings improves learning performance but increases computational cost.
- Overall, this project demonstrates how function approximation enables reinforcement learning in continuous environments and highlights the importance of feature design.